

```
=====
TEST WRITING GUIDE (RUST)
Nucleus Hardware Testing in Rust
=====
```

```
=====
OVERVIEW
=====
```

Nucleus tests are Rust `#[test]` functions using `AgentClient` SDK. The same test code runs on hardware or QEMU without changes. Tests communicate via RAM mailbox protocol – no special debug hardware needed, just the Backend trait.

Test Flow:

1. Define `[[test]]` in `stm32.toml` (name, type, script, backend)
2. Write Rust test function with `AgentClient`
3. Run: `nucleus test --backend hardware|qemu|both`
4. `AgentClient` calls → Backend trait → Hardware/QEMU execution

```
=====
PART 1: TEST PROJECT SETUP
=====
```

Directory Structure:

```
my-project/
  stm32.toml          # Test config
  src/
    main.c           # Firmware with test agent
    ...
  tests/
    my_test.rs       # Your test code
=====
```

stm32.toml Configuration:

```
[[test]]
name = "my_test"      # Human name (for --list)
type = "scripted"     # Type: "scripted" only (declarative deprecated)
script = "my_test"    # Rust test function name
backend = "both"       # "hardware" | "qemu" | "both"
=====
```

Cargo.toml Dependencies:

```
[dev-dependencies]
nucleus-test-sdk = "0.2"      # AgentClient, error types
nucleus-db = "0.2"           # Port enum, pinout database
nucleus-hil = "0.2"          # Backend trait, connect()
=====
```

```
=====
PART 2: BASIC TEST STRUCTURE
=====
```

Minimal Test (Hello World):

```
#[test]
fn hello_world() {
    eprintln!("[TEST] Hello from nucleus!");
}
```

Command: `nucleus test`

Result: `test hello_world ... [TEST] Hello from nucleus!`
`test result: ok. 1 passed`

```
=====
Test with Backend Connection:
=====
```

```
use nucleus_test_sdk::AgentClient;

#[test]
fn connect_to_device() {
    // Get backend from environment or use default
    let backend_name = std::env::var("NUCLEUS_BACKEND")
        .unwrap_or_else(|_| "hardware".to_string());
    eprintln!("[TEST] Using backend: {}", backend_name);

    // Connect to hardware or QEMU (based on stm32.toml backend field)
    let mut backend = nucleus_hil::connect(&backend_name)
        .expect("Failed to connect to backend");

    // Create agent client (talks to device firmware via RAM mailbox)
    let mut client = AgentClient::new(&mut *backend);

    // Connect: polls for magic bytes (0x4E54_4167 = 'NTAg')
    // Firmware must write magic after initializing test agent
    client.connect().expect("Failed to connect to agent");

    eprintln!("[PASS] Connected to device agent");
}
```

Command: `nucleus test --backend hardware`

Result: `Connects to NUCLEO-F411RE, waits for agent magic, continues`

```
=====
PART 3: COMMON PATTERNS
=====
```

PATTERN 1: GPIO Control & Verification

```
use nucleus_test_sdk::AgentClient;
use nucleus_db::Port;

#[test]
fn test_gpio_led() {
    let mut backend = nucleus_hil::connect("hardware").unwrap();
    let mut client = AgentClient::new(&mut *backend);
    client.connect().unwrap();

    // Set PA5 (LED) high
    eprintln!("[TEST] Setting PA5 high");
    client.set_gpio(Port::A, 5, true).unwrap();

    // Read back to verify
    let level = client.read_gpio(Port::A, 5).unwrap();
    assert!(level, "PA5 should be high");
    eprintln!("[PASS] PA5 is high");

    // Set low
    eprintln!("[TEST] Setting PA5 low");
    client.set_gpio(Port::A, 5, false).unwrap();
    let level = client.read_gpio(Port::A, 5).unwrap();
}
```

```

    assert!(level, "PA5 should be low");
    eprintln!("[PASS] PA5 is low");
}

```

Notes:

- Port::A, Port::B, ... correspond to STM32 GPIO ports
- set_gpio(port, pin, level) sets output. Blocks ~500ms timeout
- read_gpio(port, pin) reads input level (true = high, false = low)
- Both methods return Result<T, SdkError>

PATTERN 2: UART Communication

```

#[test]
fn test_uart_loopback() {
    let mut backend = nucleus_hil::connect("hardware").unwrap();
    let mut client = AgentClient::new(&mut *backend);
    client.connect().unwrap();

    // Send byte via UART
    eprintln!("[TEST] Sending 'X' via UART");
    client.uart_tx(b'X').unwrap();

    // Poll for echo (firmware must echo bytes back)
    eprintln!("[TEST] Waiting for echo...");
    let mut received = None;
    for _ in 0..10 { // Poll up to 10 times (500ms each = 5s total)
        match client.uart_rx_poll().unwrap() {
            Some(byte) => {
                received = Some(byte);
                break;
            }
            None => {
                eprintln!("[TEST] No data yet, waiting...");
                std::thread::sleep(std::time::Duration::from_millis(100));
            }
        }
    }

    assert_eq!(received, Some(b'X'), "Should receive echo");
    eprintln!("[PASS] Echo received: {}", received.unwrap() as char);
}

```

Notes:

- uart_tx(byte) sends one byte. Blocks until agent posts DONE
- uart_rx_poll() returns Option<u8>: Some(byte) or None if no data
- Agent FIFO is polled, no buffering. Check frequently or use delays
- Works on both hardware USART2 and QEMU simulated UART

PATTERN 3: Register Access

```

#[test]
fn test_register_read() {
    let mut backend = nucleus_hil::connect("hardware").unwrap();
    let mut client = AgentClient::new(&mut *backend);
    client.connect().unwrap();

    // Read GPIOA IDR register (input data register)
    // Address: 0x4002_0010 (STM32F4 memory map)
    let reg_value = client.read_register(0x40020010).unwrap();
    eprintln!("[PASS] GPIOA IDR = 0x{:08x}", reg_value);

    // Parse bits or compare against expected value
    let pa5_bit = (reg_value >> 5) & 1;
    eprintln!("[INFO] PA5 pin state: {}", pa5_bit);
}

```

Notes:

- read_register(addr) reads 32-bit value from memory
- Useful for checking hardware state, peripheral registers, etc
- Address must be valid on target (check reference manual)
- Agent does bounds checking; invalid addresses return Err

PATTERN 4: Multi-Step Test with State

```

#[test]
fn test_blink_sequence() {
    let mut backend = nucleus_hil::connect("hardware").unwrap();
    let mut client = AgentClient::new(&mut *backend);
    client.connect().unwrap();

    // Sequence: toggle LED 5 times, verify each state
    for toggle_num in 0..5 {
        let expected_state = (toggle_num % 2) == 0;

        eprintln!("[TEST] Toggle #{}: expect PA5 = {}",
            toggle_num, expected_state);

        // Firmware toggles LED every 500ms in main loop
        // We just read the current state
        let actual_state = client.read_gpio(nucleus_db::Port::A, 5).unwrap();

        assert_eq!(actual_state, expected_state,
            "Toggle {} state mismatch", toggle_num);

        eprintln!("[PASS] State verified");
        std::thread::sleep(std::time::Duration::from_millis(600));
    }
}

```

PART 4: ERROR HANDLING

SdkError Types:

```
use nucleus_test_sdk::SdkError;
```

```

#[test]
fn test_with_error_handling() {
    let mut backend = nucleus_hil::connect("hardware").unwrap();
    let mut client = AgentClient::new(&mut *backend);

    // connect() can fail with:
    match client.connect() {
        Ok(_) => eprintln!("[PASS] Connected"),
        Err(SdkError::BadMagic) => panic!("Agent not initialized"),
    }
}

```

```

Err(SdkError::VersionMismatch) => panic!("Protocol mismatch"),
Err(SdkError::Timeout) => panic!("Agent unresponsive (>500ms)",),
Err(SdkError::Hil(e)) => panic!("Backend error: {}", e),
Err(e) => panic!("Other error: {}", e),
}

// set_gpio() can fail with:
match client.set_gpio(nucleus_db::Port::A, 5, true) {
    Ok(_) => eprintln!("[PASS] GPIO set"),
    Err(SdkError::AgentError) => panic!("Agent command failed"),
    Err(SdkError::Timeout) => panic!("Agent didn't respond"),
    Err(e) => panic!("Error: {}", e),
}
}

```

Common Errors:

- BadMagic: Firmware test agent not started, or firmware not running
- VersionMismatch: SDK / firmware protocol mismatch (rebuild firmware)
- Timeout: Agent took >500ms to respond (check firmware, QEMU slowness)
- AgentError: Command failed on device (invalid port, etc)
- Hil: Backend error (ST-Link disconnected, QEMU failed to start)

PART 5: ENVIRONMENT & DEBUGGING

Logging from Test:

```

#[test]
fn test_with_logs() {
    eprintln!("[INFO] This is printed to stderr");
    println!("This is printed to stdout (captured by test runner)");

    // Run with: nucleus test --nocapture
    // Output will show in real-time (otherwise captured)
}

```

Detecting Backend (Hardware vs QEMU):

```

#[test]
fn test_backend_aware() {
    let backend_name = std::env::var("NUCLEUS_BACKEND")
        .unwrap_or_else(|_| "hardware".to_string());

    // Skip test or run different assertions based on backend
    if backend_name == "qemu" {
        eprintln!("[INFO] Running on QEMU - skipping hardware-only test");
        return; // Skip
    }

    eprintln!("[INFO] Running on {}", backend_name);
    // Continue with hardware-specific test
}

```

Alternatively, use backend field in stm32.toml:

```

[[test]]
backend = "hardware"    # Only run on hardware

[[test]]
backend = "qemu"        # Only run on QEMU

```

Timing & Delays:

```

use std::time::Duration;
use std::thread;

#[test]
fn test_with_timing() {
    let mut backend = nucleus_hil::connect("hardware").unwrap();
    let mut client = AgentClient::new(&mut *backend);
    client.connect().unwrap();

    // Firmware toggles LED every 500ms
    // Wait to observe change
    eprintln!("[TEST] Waiting 600ms for LED change...");
    thread::sleep(Duration::from_millis(600));

    let state = client.read_gpio(nucleus_db::Port::A, 5).unwrap();
    eprintln!("[PASS] Read state: {}", state);
}

```

PART 6: BEST PRACTICES

1. Always call `client.connect()` first
 - Waits for firmware agent initialization
 - Verifies protocol version match
2. Check return types with `.unwrap()` or `match`
 - `.unwrap()` panics on error (test fails)
 - `match`/? for recovery logic
3. Add `eprintln!` markers for test flow
 - "[TEST]" for actions
 - "[PASS]" for assertions
 - "[INFO]" for debug info
 - Helps trace failures in logs
4. Use assertions, not manual comparisons
 - `assert!(condition, "message")`
 - `assert_eq!(actual, expected, "message")`
 - Better error messages on failure
5. Handle timing carefully
 - Firmware may be slow (UART, polling)
 - Add delays after operations that take time
 - QEMU can be faster or slower than hardware
6. Test both success and failure paths
 - Does GPIO respond to commands?
 - What happens on invalid port/pin?
 - Timeout handling
7. Use backend field in `stm32.toml` to optimize
 - "qemu": Fast iteration during dev

- "hardware": Validate on real silicon
 - "both": Regression testing (CI)
8. Avoid magic numbers
- Use `nucleus_db::Port` enum (`Port::A`, `Port::B`)
 - Name constants (`PA5_LED = 5`)
 - Comment unclear values

PART 7: COMPLETE EXAMPLE

Realistic Multi-Device Test:

```
use nucleus_test_sdk::AgentClient;
use nucleus_db::Port;
use std::thread;
use std::time::Duration;

#[test]
fn test_complete_gpio_and_uart() {
    // Setup
    eprintln!("[TEST] Starting GPIO + UART test");
    let backend_name = std::env::var("NUCLEUS_BACKEND")
        .unwrap_or_else(|_| "hardware".to_string());

    let mut backend = nucleus_hil::connect(&backend_name)
        .expect("Failed to connect to backend");
    let mut client = AgentClient::new(&mut *backend);

    // Phase 1: Connect
    eprintln!("[TEST] Connecting to device agent...");
    client.connect().expect("Failed to connect to agent");
    eprintln!("[PASS] Connected");

    // Phase 2: Test GPIO
    eprintln!("[TEST] Testing GPIO on PA5");
    client.set_gpio(Port::A, 5, true).unwrap();
    let state = client.read_gpio(Port::A, 5).unwrap();
    assert!(state, "PA5 should be high");
    eprintln!("[PASS] GPIO set/read works");

    // Phase 3: Test UART
    eprintln!("[TEST] Testing UART loopback");
    let test_byte = b'T';
    client.udp_tx(test_byte).unwrap();

    // Poll for echo (with timeout)
    let mut echo_received = false;
    for attempt in 0..5 {
        match client.udp_rx_poll().unwrap() {
            Some(byte) => {
                assert_eq!(byte, test_byte, "Echo mismatch");
                eprintln!("[PASS] Received echo on attempt {}", attempt);
                echo_received = true;
                break;
            }
            None => {
                eprintln!("[TEST] No echo yet, waiting...");
                thread::sleep(Duration::from_millis(100));
            }
        }
    }
    assert!(echo_received, "Never received UART echo");

    // Phase 4: Combined test
    eprintln!("[TEST] Running combined GPIO toggle + UART log");
    for i in 0..3 {
        let val = (i % 2) == 0;
        client.set_gpio(Port::A, 5, val).unwrap();
        let msg = format!("Toggle {} = {}\n", i, if val { "high" } else { "low" });
        client.udp_tx(msg.as_bytes().first().unwrap()).unwrap();
        eprintln!("[PASS] {}", msg.trim());
        thread::sleep(Duration::from_millis(50));
    }

    eprintln!("[PASS] All tests passed!");
}

stm32.toml:
[[test]]
name = "complete_test"
type = "scripted"
script = "test_complete_gpio_and_uart"
backend = "both"
```

RUNNING TESTS

```
List all tests:
$ nucleus test --list

Run all tests:
$ nucleus test

Run specific test:
$ nucleus test my_test_name

Run on hardware only:
$ nucleus test --backend hardware

Run on QEMU only:
$ nucleus test --backend qemu

Run with output:
$ nucleus test --nocapture

Run and show timings:
$ nucleus test --test-threads=1

Run for CI (fast):
$ nucleus test --backend qemu # Use QEMU in CI for speed
$ nucleus test --backend hardware # Use hardware for release validation
```